

Databricks Interview Guide: Zero to Mid-Level

A Comprehensive Playbook for Data Engineering Interviews

INTRODUCTION: WHAT IS DATABRICKS?

1. What is Databricks in simple terms?

Databricks is a cloud-based platform for processing, storing, cleaning, and analyzing massive amounts of data. It is built on top of **Apache Spark**. Think of it as a "Unified Environment" where Data Engineers, Data Scientists, and Analysts all work in the same space.

- **Core Engine:** Apache Spark (Distributed computing).
- **Storage Format:** Delta Lake (Parquet with "superpowers").
- **Architecture:** The "Lakehouse" (combines Data Lake flexibility with Data Warehouse reliability).

2. OLTP vs. OLAP: Where does Databricks live?

- **OLTP (Online Transactional Processing):** Databases like MySQL or PostgreSQL used for daily transactions (e.g., swiping a credit card).
- **OLAP (Online Analytical Processing):** Platforms like Databricks used for big data analysis, reporting, and ML.

3. Why choose Databricks over Snowflake or Hadoop?

Compared to Hadoop, Databricks is much faster and managed (you don't maintain servers). Compared to Snowflake, Databricks handles "Unstructured" data (images, logs) and Machine Learning much better.

WORKING WITH DATA: READING & WRITING

4. How to Read and Write Data in PySpark?

In an interview, you must know the syntax for different formats.

```
# Reading CSV with Schema Inference
df = spark.read.format("csv").option("header", "true").option("inferSchema", "true").load("/path/to/file.csv")

# Writing to Delta (The Databricks Default)
df.write.format("delta").mode("overwrite").save("/mnt/delta/my_table")

# Creating a Table in the Metastore (so you can query via SQL)
df.write.format("delta").saveAsTable("my_database.my_table")
```

5. What are ACID Properties and why do we need them?

Standard Data Lakes (S3) are not ACID-compliant. If a write fails halfway, you get "dirty" data. Delta Lake brings ACID:

- **Atomicity:** The write either fully succeeds or fully fails.
- **Consistency:** Prevents data corruption.
- **Isolation:** Multiple people can read/write at the same time without seeing partial data.
- **Durability:** Data is saved permanently on cloud storage (S3/ADLS).

CLOUD CONNECTIVITY: CONNECTING TO S3 / AZURE

6. How do you connect Databricks to AWS S3 or Azure ADLS?

There are three main ways to bring data into Databricks:

1. **Access Keys (Insecure):** Hardcoding keys in the notebook.
2. **Mounting (Old Way):** Using `dbutils.fs.mount` to make S3 look like a local folder.
3. **Unity Catalog / IAM Roles (Best Practice):** Creating an "External Location" in Unity Catalog. You attach a "Storage Credential" to the Databricks cluster, so the cluster can "talk" to S3 without you typing any passwords.

7. Code Example: Connecting to S3 using Access Keys

```
spark.conf.set("fs.s3a.access.key", dbutils.secrets.get(scope="aws", key="access_key"))
spark.conf.set("fs.s3a.secret.key", dbutils.secrets.get(scope="aws", key="secret_key"))
df = spark.read.parquet("s3a://my-bucket-name/data/")
```

8. Narrow vs. Wide Transformations (Crucial Interview Question!)

This determines the speed of your code.

- **Narrow:** Data stays within one machine (Executor). Operations: `select()`, `filter()`, `union()`. These are very fast.
- **Wide:** Data must be moved across the network to other machines. This is called a ****Shuffle****. Operations: `groupBy()`, `join()`, `distinct()`, `repartition()`. These are slow.

9. Coalesce vs. Repartition: Which one to use?

- **Repartition:** Used to ****increase or decrease**** partitions. It causes a full shuffle. Use it to fix "Data Skew."
- **Coalesce:** Used only to ****decrease**** partitions. It is much faster because it avoids a full shuffle. Use it just before writing your final output to S3 to reduce the number of files.

10. What is Data Skew and how do you fix it?

Skew is when 90% of your data lives in one partition (e.g., joining on a "Country" column where "India" has 1 billion rows and "Vatican" has 10). **The Fix:** Use ****Salting****. You add a random number (salt) to the join key to spread the data across executors.

```
# Salting example
from pyspark.sql.functions import rand, lit
df_salted = df.withColumn("salt", (rand() * 10).cast("int"))
```

11. Broadcast Join: The Performance King.

If you are joining a tiny table (e.g., 5MB) with a huge table (e.g., 10TB), Spark can send the tiny table to *every* machine. This avoids a shuffle.

```
from pyspark.sql.functions import broadcast
joined_df = big_df.join(broadcast(small_df), "id")
```

DATA MODELING: SCD TYPE 1 AND TYPE 2

12. Explain SCD Type 1 vs Type 2.

- **SCD Type 1 (Overwrite):** You just update the record. You lose history. (e.g., Changing a user's phone number).
- **SCD Type 2 (History):** You add a new row with a new version. You keep history using `is_current` or `start_date/end_date` columns.

13. Implementing Upserts (MERGE) in Delta Lake.

In standard Spark, you can't update a single row. In Delta Lake, you use `MERGE`.

```
from delta.tables import DeltaTable
deltaInstance = DeltaTable.forPath(spark, "/path/to/table")
deltaInstance.alias("target").merge(
    source_df.alias("source"), "target.id = source.id"
).whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()
```

GOVERNANCE: UNITY CATALOG

14. What is Unity Catalog and why is it used?

Unity Catalog is the "Security and Governance" layer. It provides:

- **Centralized Permissions:** You manage access to all your tables in one place.
- **Data Lineage:** It shows you exactly where data came from (Bronze → Silver → Gold).
- **Auditing:** You can see who accessed what data and when.

15. Explain the 3-Tier Namespacing in Unity Catalog.

Before, we just had `database.table`. Now we have: `catalog.schema.table`. Example: `production.sales_data.customer_records`

16. How to handle Nested JSON Data?

In interviews, you will often be asked how to deal with API responses. Use `explode()` to flatten arrays.

```
from pyspark.sql.functions import explode
# If 'friends' is an array column, this creates a new row for every friend.
df_flat = df.withColumn("friend_name", explode(df.friends))
```

17. Window Functions: Finding the "Latest" record.

A very common mid-level question: "How do you find the most recent transaction for every user?"

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, desc

windowSpec = Window.partitionBy("user_id").orderBy(desc("transaction_date"))
df_latest = df.withColumn("rank", row_number().over(windowSpec)).filter("rank == 1")
```

18. What is "Caching" and when should you use it?

If you are going to use the same DataFrame multiple times (e.g., joined in 3 different places), use `df.cache()`. This stores the data in memory so Spark doesn't have to re-read it from S3 three times.

ECOSYSTEM: WORKFLOWS, DLT, AND PHOTON

19. What are Databricks Workflows?

This is the internal scheduler (like Airflow). You can create a "Job" that runs a Notebook every morning. It supports "Retry" logic and dependencies.

20. What is Delta Live Tables (DLT)?

DLT is a framework that simplifies ETL. Instead of writing complex Spark code, you define your tables as "Live Tables," and Databricks handles the orchestration, error handling, and data quality checks automatically.

21. Explain Photon Engine.

Photon is a specialized query engine within Databricks written in C++. It makes Spark queries much faster by using "Vectorized Execution." You don't write code for Photon; Databricks uses it automatically if the cluster supports it.

PERFORMANCE TUNING & DEBUGGING

22. What is the Spark UI and how do you use it?

The Spark UI is your "Doctor's Stethoscope."

- **Stages Tab:** Shows where the job is stuck.
- **SQL Tab:** Shows the "Query Plan." Look for "Scan" (reading data) and "Exchange" (shuffling data).
- **Storage Tab:** See how much data is cached in memory.

23. Z-Ordering: How does it improve performance?

Z-Ordering is like an index. It physically rearranges the data so that related information is stored in the same file. **Best Use Case:** Columns that you use frequently in WHERE clauses (like `order_id`).

```
OPTIMIZE my_table ZORDER BY (customer_id);
```

24. Partitioning vs Z-Ordering: When to use which?

- **Partitioning:** Use for columns with low cardinality (e.g., Year, Month, Region).
- **Z-Ordering:** Use for columns with high cardinality (e.g., User_ID, Order_ID).

Scenario A: Your Spark job is failing with an "Out Of Memory (OOM)" error.

Possible Causes:

1. **Driver OOM:** You tried to `.collect()` too much data back to the driver.
2. **Executor OOM:** Your partitions are too large, or you have a massive data skew.

The Fix: Use `repartition()` to increase the number of partitions (making each one smaller) or increase the worker node size.

Scenario B: Your query is slow on a 10TB table even though you have a powerful cluster.

Checklist:

- Are you reading thousands of small files? (Run `OPTIMIZE`).
- Are you shuffling 10TB of data? (Check for wide transformations).
- Are you using a Broadcast join for the small tables?

Scenario C: You accidentally deleted some records from a Delta table.

Solution: Use `**Time Travel**`.

```
-- Query the table as it was 1 hour ago
SELECT * FROM my_table TIMESTAMP AS OF '2025-01-01 10:00:00';
```

PROJECT LIFECYCLE: FROM BRONZE TO GOLD

25. Describe a typical Data Pipeline in Databricks.

- **Ingestion (Bronze):** Landing raw files from S3/Kafka. Data is "Dirty."
- **Transformation (Silver):** Cleaning data. Handling nulls, removing duplicates, joining with lookup tables.
- **Aggregation (Gold):** Creating final tables for the Business (e.g., Total Sales per Region). This is where Power BI or Tableau connects.

26. CI/CD in Databricks: How do you deploy code?

We use `**Databricks Repos**` to sync notebooks with GitHub. We then use `**Azure DevOps**` or `**GitHub Actions**` to automatically deploy code from the "Dev" workspace to the "Prod" workspace using the Databricks CLI.

INTERVIEW TIPS: CLOSING STRONG

- **Keywords to use:** Shuffling, Data Skew, Partition Pruning, Medallion Architecture, ACID Transactions.
- **The "Cost" Angle:** Always mention that you use **Spot Instances** or **Job Clusters** to save the company money.
- **Security:** Mention that you use `**Secret Scopes**` so that passwords are never visible in the code.